

9주차 · 10/29(목)

CSS3 고급 활용

Flexbox, Grid, 반응형 웹 디자인 기초

한신대학교 AI·SW계열 · 웹프로그래밍
학습사이트 hs-web.dreamitbiz.com

학습 목표

- ① Flexbox의 주축·교차축 개념을 이해하고 정렬 속성을 정확히 쓸 수 있다.
- ② Grid로 2차원 격자를 만들고 minmax·auto-fit을 활용할 수 있다.
- ③ 미디어쿼리로 화면 폭에 따라 배치를 바꿀 수 있다.
- ④ 미디어쿼리가 뒤 규칙에 덮여 죽는 문제를 진단하고 고칠 수 있다.
- ⑤ CSS 변수로 색과 여백을 한 곳에서 관리할 수 있다.

9주차

핵심 키워드

flex

justify-content

align-items

gap

grid

minmax

auto-fit

미디어쿼리

모바일 우선

CSS 변수

학습사이트의 본문에서 이 낱말들을 찾아 읽으면 이 시간의 뼈대가 잡힙니다

9주차 · 1

Flexbox — 한 줄 정렬의 표준

Flexbox는 한 방향(가로 또는 세로)으로 요소를 늘어놓고 정렬하는 도구입니다.

커는 곳은 자식이 아니라 부모입니다

```
1  .container {          /* ← 부모(flex container)에 선언 */
2    display: flex;
3  }
4  /* .container의 자식들(flex item)이 자동으로 가로로 늘어섭니다. */
```

내비게이션 메뉴, 카드 나열, 버튼 묶음, 가운데 정렬 — 실무에서 가장 많이 쓰는 배치 도구입니다.

1.1 주축과 교차축 — 이것만 이해하면 됩니다

- 1 Flexbox에는 두 개의 축이 있습니다.
- 2 주축(main axis) 은 요소가 늘어서는 방향, 교차축(cross axis) 은 그것과 직각인 방향입니다.
- 3 flex-direction 이 주축을 정합니다.

1.1 주축과 교차축 — 이것만 이해하면 됩니다

flex-direction	주축	교차축	justify-content가 움직이는 방향
row (기본값)	가로 →	세로 ↓	가로
column	세로 ↓	가로 →	세로
row-reverse	가로 ←	세로 ↓	가로(역방향)

헛갈릴 때 이렇게 기억하세요

- 1 **justify-content = 주축**
을 따라 정렬 (요소가 늘어서는 방향)
- 2 **align-items = 교차축**
을 따라 정렬 (직각 방향)
- 3 **flex-direction: column** 이면 둘의 방향이 뒤바뀝니다. 안 먹는 것 같으면 **direction**부터 확인하세요.

flex container 속성 (1/2)

```

1  .container {
2      display: flex;
3
4      flex-direction: row;      /* row | column | row-reverse | column-reverse */
5
6      justify-content: center;  /* 주축 정렬
7      flex-start      ■■■□□□□□□□  왼쪽 몰기 (기본)
8      center          □□□■ ■ ■ □□□□  가운데
9      flex-end        □□□□□□■ ■ ■   오른쪽 몰기
10     space-between   ■□□■□□□□■     양끝 붙이고 사이 균등 ← 헤더에 많이 씀
11     space-around    □■□□■□□■□     각자 좌우 균등

```

flex container 속성 (2/2)

```
19     space-evenly  □■□■□■□   간격 완전 균등 */
20
21     align-items: center;      /* 교차축 정렬
22     stretch(기본) | flex-start | center | flex-end | baseline */
23
24     flex-wrap: wrap;          /* 넘치면 다음 줄로 (기본은 nowrap: 찌그러짐) */
25
26     gap: 16px;                /* 요소 사이 간격. margin보다 훨씬 편하다 */
27     gap: 16px 24px;           /* 행 간격 열 간격 */
28 }
```

완벽한 가운데 정렬 — 세 줄이면 끝

```
1  .center {
2    display: flex;
3    justify-content: center; /* 가로 가운데 */
4    align-items: center;    /* 세로 가운데 */
5    height: 300px;
6  }
7
8  /* Flexbox 이전에는 이 간단한 것을 하려고
9     온갖 편법을 써야 했습니다. 지금은 세 줄입니다. */
```

flex item 속성

```
1  .item {
2    flex-grow: 1;      /* 남은 공간을 나눠 가진다 (0 = 안 늘어남) */
3    flex-shrink: 1;    /* 공간이 모자라면 줄어든다 (0 = 안 줄어듦) */
4    flex-basis: 220px; /* 기본 크기 */
5
6    flex: 1 1 220px;   /* 위 셋을 한 줄로 (grow shrink basis) */
7    flex: 1;           /* = flex: 1 1 0 — "균등하게 나눠 가져라" */
8
9    align-self: flex-end; /* 이 요소만 교차축 정렬을 다르게 */
10   order: 2;           /* 순서 바꾸기 (HTML은 그대로) */
11 }
```

1.3 자식에 주는 속성

자주 쓰는 조합	뜻
<code>flex: 1</code>	남는 공간을 균등하게 나눠 가진다
<code>flex: 1 1 220px</code>	기본 220px인데 늘어나도 줄어도 된다 — 반응형 카드
<code>flex: 0 0 200px</code>	무슨 일이 있어도 200px 고정 — 사이드바
<code>margin-left: auto</code>	이 요소부터 오른쪽 끝으로 밀기 — 헤더의 로그인 버튼

기본값 nowrap의 함정

```
1  /* ❌ 좁은 화면에서 카드가 찌그러진다 */
2  .cards { display: flex; gap: 16px; }
3
4  /* ✅ 넘치면 다음 줄로 내려간다 */
5  .cards { display: flex; flex-wrap: wrap; gap: 16px; }
6  .cards > * { flex: 1 1 220px; } /* 220px 아래로는 안 줄어들고 줄바꿈 */
```

1.4 flex-wrap을 빠뜨리면 생기는 일

- 1 Flexbox에 flex-wrap: wrap 과 flex-basis 를 함께 주면 미디어쿼리 없이도 반응형이 됩니다.
- 2 6주차에서 Grid의 auto-fit 과 함께 다시 다룹니다.

9주차 · 2

Flexbox 실전 패턴

① 헤더 — 로고 왼쪽, 메뉴 오른쪽

```
1 <header class="nav">
2   <strong class="logo">MySite</strong>
3   <nav>
4     <ul class="menu">
5       <li><a href="#">소개</a></li>
6       <li><a href="#">작업</a></li>
7       <li><a href="#">연락</a></li>
8     </ul>
9   </nav>
10 </header>
```

① 헤더 — CSS (1/2)

```
1  .nav {
2    display: flex;
3    justify-content: space-between; /* 양끝으로 */
4    align-items: center;           /* 세로 가운데 */
5    padding: 0 20px;
6    height: 60px;
7    border-bottom: 1px solid #e5e7eb;
8  }
9
10 .menu {
11   display: flex;
```

① 헤더 — CSS (2/2)

```
19     gap: 20px;
20     list-style: none;    /* △ 점(·) 없애기 */
21     padding: 0;         /* △ ul의 기본 왼쪽 여백 없애기 */
22     margin: 0;
23 }
24
25 .menu a {
26     text-decoration: none;
27     color: #4b5563;
28 }
29 .menu a:hover { color: #4338ca; }
```

② 미디어 오브젝트 — 이미지 옆에 글

```
1  .media {  
2    display: flex;  
3    gap: 16px;  
4    align-items: flex-start; /* 위쪽 정렬 */  
5  }  
6  .media img {  
7    flex: 0 0 80px; /* 80px 고정 — 안 늘고 안 줄고 */  
8    border-radius: 50%;  
9  }  
10 .media .body { flex: 1; } /* 나머지 전부 */
```

③ 푸터를 화면 아래에 붙이기 (내용이 짧아도)

```
1 body {  
2   display: flex;  
3   flex-direction: column;  
4   min-height: 100vh; /* 화면 높이만큼 최소 확보 */  
5   margin: 0;  
6 }  
7 main { flex: 1; } /* 본문이 남는 공간을 다 차지 → 푸터가 아래로 */
```


흔한 실수 모음

- 1 display: flex 를 자식에 줬다**
부모에 줘야 합니다.
- 2 flex-wrap 을 안 줘서**
좁은 화면에서 다 찢그러진다.
- 3 ul 의 list-style: none; padding: 0; 을 빠뜨려서**
점과 들여쓰기가 남는다.
- 4 align-items 와 justify-content 를 바꿔 썼다**
direction이 column이면 뒤바뀝니다.
- 5 gap 대신 margin 을 써서**
맨 끝에도 여백이 남는다. gap을 쓰세요.

9주차 · 3

Grid — 행과 열이 있는 2차원 배치

Flexbox가 한 줄을 다루는 도구라면, Grid는 행과 열이 모두 있는 격자를 다루는 도구입니다.

Grid — 행과 열이 있는 2차원 배치

	Flexbox	Grid
차원	1차원 (한 방향)	2차원 (행 + 열)
기준	내용 중심 — 내용이 크기를 정한다	틀 중심 — 격자를 먼저 정하고 채운다
잘 맞는 곳	내비게이션, 버튼 묶음, 카드 한 줄	갤러리, 대시보드, 페이지 전체 뼈대
실무	더 자주 쓴다	큰 틀에 쓴다

사진 갤러리, 대시보드, 페이지 전체 레이아웃처럼 가로세로가 모두 중요한 곳에 씁니다.

Grid — 행과 열이 있는 2차원 배치

- 1 둘은 경쟁 관계가 아닙니다.
- 2 큰 틀은 Grid로 잡고, 각 칸 안은 Flexbox로 정렬하는 조합이 가장 흔합니다.

Grid 켜기

```
1  .grid {  
2    display: grid;  
3    grid-template-columns: 200px 1fr 200px; /* 3열: 고정 · 남는 만큼 · 고정 */  
4    grid-template-rows: auto 1fr auto; /* 3행 */  
5    gap: 16px;  
6  }
```

fr은 "fraction(조각)"입니다

```
1 grid-template-columns: 1fr 1fr 1fr;      /* 균등 3등분 */
2 grid-template-columns: repeat(3, 1fr);    /* 위와 같음 — repeat로 줄여 쓰기 */
3 grid-template-columns: 2fr 1fr;          /* 2:1 비율 */
4 grid-template-columns: 200px 1fr;        /* 왼쪽 고정, 오른쪽 나머지 전부 */
```

실무에서 반드시 알아야 할 함정

```
1  /* ❌ 안에 든 것이 크면 칸이 안 줄어든다 */
2  grid-template-columns: repeat(3, 1fr);
3
4  /* ✅ 내용이 커도 칸이 줄어든다 */
5  grid-template-columns: repeat(3, minmax(0, 1fr));
6
7  /* 왜? 1fr의 최소 크기는 auto(=내용 크기)입니다.
8     긴 URL이나 넓은 표가 안에 들어가면 그 칸이 안 줄어들어
9     격자가 통째로 화면 밖으로 빠져나갑니다.
10    minmax(0, 1fr)은 최소를 0으로 못 막아 이걸 막습니다. */
```

미디어쿼리 없는 반응형 격자

```
1 .gallery {
2   display: grid;
3   grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
4   gap: 16px;
5 }
6
7 /* 읽는 법: "240px보다 작아지지 않는 칸을, 들어가는 만큼 만들어라"
8    → 화면이 1000px이면 4열, 600px이면 2열, 300px이면 1열.
9    브레이크포인트를 하나도 안 썼는데 반응형입니다. */
```


3.4 auto-fit — 그 자체가 반응형

- 1 auto-fit 과 auto-fill 은 비슷하지만, 항목이 적을 때 auto-fit 은 남은 칸을 접어 항목을 늘리고, auto-fill 은 빈 칸을 남겨 둡니다.
- 2 대부분 auto-fit 이 원하는 동작입니다.
- 3 `repeat(auto-fit, minmax(...))` 에 미디어쿼리를 덧붙이지 마세요.
- 4 이미 반응형인데 브레이크포인트를 없으면 오히려 동작이 이상해집니다.

span과 grid-template-areas

```
1  /* 칸 합치기 */
2  .wide { grid-column: span 2; }    /* 가로 2칸 차지 */
3  .tall { grid-row: span 2; }      /* 세로 2칸 차지 */
4
5  /* 영역에 이름 붙이기 — 페이지 뼈대에 아주 편합니다 */
6  .layout {
7    display: grid;
8    grid-template-columns: 200px 1fr;
9    grid-template-areas:
10     "header header"
11     "side  main"
12     "footer footer";
13    gap: 16px;
14  }
15  .layout > header { grid-area: header; }
16  .layout > aside { grid-area: side; }
17  .layout > main { grid-area: main; }
18  .layout > footer { grid-area: footer; }
```

9주차 · 4

반응형 웹 — 하나의 코드로 모든 화면

사람들이 웹을 보는 화면은 320px 휴대폰부터 27인치 모니터까지 제각각입니다.

반응형 웹 — 하나의 코드로 모든 화면

- 1 반응형 웹은 화면 폭에 따라 배치가 알아서 바뀌게 만드는 것입니다.

viewport 메타 태그

```
1 <meta name="viewport" content="width=device-width, initial-scale=1">
2
3 <!-- 이 한 줄이 없으면 휴대폰이 "이 페이지는 980px짜리 데스크톱 페이지겠지"
4      하고 축소해서 보여 줍니다. 그러면 미디어쿼리가 아예 발동하지 않아
5      반응형 CSS를 아무리 써도 소용이 없습니다.
6      "폰에서 글자가 개미만 해요"의 원인 1위입니다. -->
```

화면 조건에 따라 다른 규칙 (1/2)

```
1  /* 기본(모든 화면) */
2  .grid { grid-template-columns: repeat(3, minmax(0, 1fr)); }
3
4  /* 화면이 768px 이하일 때만 */
5  @media (max-width: 768px) {
6    .grid { grid-template-columns: 1fr; }
7  }
8
9  /* 범위로도 쓸 수 있습니다 */
10 @media (min-width: 769px) and (max-width: 1024px) { }
11
12 /* 인쇄할 때 */
```

화면 조건에 따라 다른 규칙 (2/2)

```
19 @media print {  
20   .navbar, .footer { display: none; }  
21 }  
22  
23 /* 기기의 다크모드 설정 */  
24 @media (prefers-color-scheme: dark) { }  
25  
26 /* 움직임을 줄여 달라는 설정 (멀미 · 발작 위험이 있는 사용자) */  
27 @media (prefers-reduced-motion: reduce) {  
28   * { animation: none !important; transition: none !important; }  
29 }
```

4.3 자주 쓰는 브레이크포인트

폭	대상	보통 하는 일
~480px	작은 휴대폰	1열, 메뉴는 햄버거로
~768px	휴대폰·작은 태블릿	1~2열
~1024px	태블릿·작은 노트북	2~3열
1025px~	데스크톱	전체 배치

4.3 자주 쓰는 브레이크포인트

- 1 이 숫자는 법이 아닙니다.
- 2 내 디자인이 깨지는 지점이 곧 내 브레이크포인트입니다.
- 3 브라우저 창을 천천히 줄여 보다가 어색해지는 폭에서 끊는 것이 가장 좋습니다.

두 가지 방향

```
1  /* 방식 ① 데스크톱 우선 — 기본은 넓은 화면, max-width로 줄여 감 */
2  .grid { grid-template-columns: repeat(3, minmax(0, 1fr)); }
3  @media (max-width: 768px) { .grid { grid-template-columns: 1fr; } }
4
5  /* 방식 ② 모바일 우선 — 기본은 좁은 화면, min-width로 넓혀 감 (권장) */
6  .grid { grid-template-columns: 1fr; }
7  @media (min-width: 769px) { .grid { grid-template-columns: repeat(3, minmax(0, 1fr)); } }
```

4.4 모바일 우선(Mobile First)

- 1 모바일 우선을 권하는 이유 — 대부분의 방문자가 휴대폰으로 옵니다.
- 2 기본 스타일이 모바일이면 휴대폰은 미디어쿼리를 읽을 필요조차 없어 조금 더 빠르고, 무엇보다 "좁은 화면에서 어떻게 보일지"를 먼저 생각하게 되어 결과가 좋아집니다.

9주차 · 5

⚠ 미디어쿼리에는 우선권이 없습니다

이것은 실무에서 가장 자주, 가장 조용히 사람을 괴롭히는 문제입니다.

모바일 규칙이 죽는 순간

```
1  /* style.css 589번째 줄 */
2  @media (max-width: 768px) {
3    .pg-grid { grid-template-columns: 1fr; }
4  }
5
6  /* style.css 623번째 줄 */
7  .pg-grid { grid-template-columns: repeat(2, 1fr); }  /* ← 이게 이겁니다 */
8
9  /* 결과: 휴대폰에서도 2열 그대로.
10     화면에는 "좁은 화면에서 안 접힌다"로만 보이고,
11     빌드는 아무 오류 없이 통과합니다. */
```

실제로 여러 사이트를 전수 점검했을 때 65개 사이트 중 18건이 이 형태였습니다.

왜 이런 일이 생기는가

- 1 **@media 안에 있다고 우선권이 생기지 않습니다.**
특정도가 같으면 나중에 온 쪽이 이깁니다 — 4주차 캐스케이드 그대로입니다.
- 2 **해결: 반응형 규칙을 파일 맨 뒤에**
둡니다. 파일을 나눴다면 responsive.css 를 마지막에 불러옵니다.
- 3 **진단: 개발자도구 Elements → Styles**
에서 취소선이 그어진 규칙을 찾습니다. 그게 진 규칙입니다.
- 4 **responsive.css 가 main.css 보다 앞에 있으면 이름과 달리 반응형이 아닙니다.**

9주차 · 6

좁은 화면에서 넘치는 것들

이미지 반응형의 기본

```
1  img {  
2    max-width: 100%; /* 부모보다 커지지 않는다 */  
3    height: auto; /* 비율 유지 */  
4    display: block; /* 아래 생기는 정체불명의 4px 여백 제거 */  
5  }
```


표는 감싸서 스크롤시킵니다

```
1 <div class="table-scroll">
2   <table>...</table>
3 </div>
```

표 — CSS

```
1 .table-scroll {  
2     overflow-x: auto;           /* 표만 가로로 스크롤 */  
3     -webkit-overflow-scrolling: touch;  
4 }  
5 .table-scroll table { min-width: 480px; }  
6  
7 /* △ body에 가로 스크롤이 생기게 두면 안 됩니다.  
8     넘치는 것은 그 요소 안에서만 스크롤되게 합니다. */
```

한국어 + 긴 URL

```
1 p, li, td {  
2   word-break: keep-all;      /* 어절 중간에서 안 끊는다 */  
3   overflow-wrap: break-word; /* 공백 없는 긴 URL만 예외로 꺾는다 */  
4 }
```


좌우 여백을 섹션마다 하드코딩하지 않기

```
1  :root {
2    --pad: 20px;
3  }
4  @media (min-width: 640px) { :root { --pad: 28px; } }
5  @media (min-width: 1024px) { :root { --pad: 40px; } }
6
7  .inner {
8    max-width: calc(1200px + 2 * var(--pad)); /* 내용 폭 + 좌우 패딩 */
9    padding: 0 var(--pad);
10   margin: 0 auto;
11  }
12
13  /* △ max-width에 패딩을 포함시키지 마세요.
14     box-sizing: border-box 이면 패딩이 바뀔 때 내용 폭까지 흔들립니다. */
```

9주차 · 7

반응형은 눈으로 확인해야 합니다

점검 방법

- 1 개발자도구의 기기 도구 모음**
(F12 → 왼쪽 위 휴대폰 아이콘)으로 360·390·768·1024px을 차례로 봅니다.
- 2 가로 스크롤이 생기는지**
반드시 확인합니다. 생겼다면 어떤 요소가 넘치는지 Elements에서 하나씩 눌러 찾습니다.
- 3 창을 마우스로 천천히 줄여 보세요.**
어색해지는 순간이 곧 브레이크포인트를 넣을 자리입니다.
- 4 브라우저 창 크기를 직접 조절하는 것은 정확하지 않습니다 — 브라우저 크롬 때문에 CSS 뷰포트와 어긋납니다. 기기 도구 모음을 쓰세요.**

9주차 · 8

CSS 변수 — 색을 한 곳에서

사이트의 주요 색을 바꾸려는데 그 색이 CSS 파일 40군데에 흩어져 있다면 어떻게 될까요?

CSS 변수 — 색을 한 곳에서

- 1 하나라도 빠뜨리면 그 자리만 옛날 색으로 남습니다.
- 2 CSS 변수(사용자 지정 속성) 는 이 문제를 없앱니다.

정의하고 꺼내 쓰기

```
1  :root {                                /* 문서 전체에서 쓸 변수는 :root에 */
2    --main: #4338ca;
3    --main-dark: #312e81;
4    --text: #111827;
5    --bg: #ffffff;
6    --radius: 12px;
7    --pad: 20px;
8  }
9
10 h1    { color: var(--main); }
11 button { background: var(--main); border-radius: var(--radius); }
12 .card { padding: var(--pad); background: var(--bg); }
13
14 /* 이제 --main 한 줄만 바꾸면 사이트 전체 색이 바뀝니다. */
```

기본값과 지역 변수

```
1  /* 변수가 없을 때 쓸 값을 함께 줄 수 있습니다 */
2  color: var(--accent, #666);
3
4  /* 특정 영역에서만 다른 값을 쓸 수도 있습니다 (상속됩니다) */
5  .dark-section {
6    --text: #f9fafb;      /* 이 안에서만 --text가 밝은 색 */
7    color: var(--text);
8  }
```

변수로 묶어 두면 좋은 것들

- 1 색**
주요 색, 글자색, 배경색, 테두리색
- 2 여백**
좌우 패딩(--pad). 화면 폭마다 값만 바꾸면 전체가 따라옵니다.
- 3 모서리·그림자**
사이트 전체의 인상을 한 번에 조절
- 4 컨테이너 폭**
--container: 1200px

CSS 변수 — 색을 한 곳에서

- 1 장식 요소도 변수에 묶으세요.
- 2 히어로의 떠 있는 칩을 `right: 14px` 로 고정해 두면, 본문 여백이 40px일 때 장식만 14px이라 어긋나 보입니다.
- 3 `right: calc(var(--pad) + 4px)` 처럼 묶어 두면 자동으로 따라옵니다.

9주차

실습

이 시간의 실습 7개 — 학습사이트의 코드 실습장에서 직접 치고 실행합니다

실습 1 · 기본

CSS · 학습사이트 실습장에서 작성 · 「확인」으로 채점

“**.box 안의 내용을 가로·세로 모두 정확히 가운데에 놓으세요. 높이는 240px, 배경은 연한 보라색으로 합니다.**”

힌트

display:flex 를 켜 뒤 **justify-content** 와 **align-items** 를 씁니다.

실습 1 모범답안

```
1  .box {  
2    display: flex;  
3    justify-content: center;  
4    align-items: center;  
5    height: 240px;  
6    background: #eef2ff;  
7    border-radius: 12px;  
8  }
```

실습 2 · 기본

CSS · 학습사이트 실습장에서 작성 · 「확인」으로 채점

“ul을 가로 내비게이션 메뉴로 만드세요. 점(•)과 기본 들여쓰기를 없애고, li를 가로로 20px 간격으로 늘어놓고, 링크의 밑줄을 없앤 뒤 마우스를 올리면 색이 바뀌게 하세요.

힌트

ul에 list-style: none 과 padding: 0 을 함께 주는 것을 잊지 마세요.

실습 2 모범답안

```
1  ul {
2    display: flex;
3    gap: 20px;
4    list-style: none;
5    padding: 0;
6    margin: 0;
7  }
8
9  ul a {
10   text-decoration: none;
11   color: #4b5563;
12   padding: 8px 4px;
13 }
14
15 ul a:hover {
16   color: #4338ca;
17 }
```

실습 3 · 심화

CSS · 학습사이트 실습장에서 작성 · 「확인」으로 채점

“카드 목록을 미디어쿼리 없이 반응형으로 만드세요. 부모는 flex에 wrap과 16px 간격을 주고, 각 카드는 기본 240px이되 늘어나거나 줄어들 수 있게 하세요. 카드에는 padding·테두리·둥근 모서리도 줍니다.

힌트

flex: 1 1 240px 한 줄이면 됩니다. 화면이 좁아지면 알아서 아래로 접힙니다.

실습 3 모범답안

```
1  .cards {  
2    display: flex;  
3    flex-wrap: wrap;  
4    gap: 16px;  
5  }  
6  
7  .card {  
8    flex: 1 1 240px;  
9    padding: 20px;  
10   border: 1px solid #e5e7eb;  
11   border-radius: 12px;  
12   background: #fff;  
13 }
```


실습 4 · 기본

CSS · 학습사이트 실습장에서 작성 · 「확인」으로 채점

“미디어쿼리 없이 반응형이 되는 갤러리 격자를 만드세요. 칸의 최소 폭은 200px이고, 화면이 넓어지면 열이 자동으로 늘어나야 합니다.

힌트

`repeat(auto-fit, minmax(200px, 1fr))` 한 줄입니다.

실습 4 모범답안

```
1  .gallery {
2    display: grid;
3    grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
4    gap: 12px;
5  }
6
7  .gallery img {
8    width: 100%;
9    aspect-ratio: 1 / 1;
10   object-fit: cover;
11   display: block;
12   border-radius: 8px;
13 }
```

실습 5 · 응용

CSS · 학습사이트 실습장에서 작성 · 「확인」으로 채점

“모바일 우선 방식으로 카드 격자를 만드세요. 기본은 1열이고, 640px 이상에서 2열, 1024px 이상에서 3열이 되게 하세요.

힌트

min-width 미디어쿼리를 좁은 것부터 넓은 것 순서로 씁니다. 순서가 중요합니다.

실습 5 모범답안

```
1  .cards {  
2    display: grid;  
3    grid-template-columns: 1fr;  
4    gap: 16px;  
5  }  
6  
7  @media (min-width: 640px) {  
8    .cards { grid-template-columns: repeat(2, minmax(0, 1fr)); }  
9  }  
10  
11 @media (min-width: 1024px) {  
12   .cards { grid-template-columns: repeat(3, minmax(0, 1fr)); }  
13 }
```

실습 6 · 심화

CSS · 학습사이트 실습장에서 작성 · 「확인」으로 채점

“아래 CSS는 모바일에서 안 접힙니다. 원인을 찾아 고치세요. (규칙은 지우지 말고 순서만 바로잡으세요.)

힌트

미디어쿼리에는 우선권이 없습니다. 특정도가 같으면 나중에 온 것이 이깁니다.

실습 6 모범답안

```
1  .grid {
2    display: grid;
3    grid-template-columns: repeat(3, minmax(0, 1fr));
4    gap: 16px;
5  }
6
7  /* 반응형 규칙은 파일 뒤쪽에 — 앞에 두면 뒤 규칙에 덮여 죽습니다 */
8  @media (max-width: 768px) {
9    .grid { grid-template-columns: 1fr; }
10 }
```

실습 7 · 기본

CSS · 학습사이트 실습장에서 작성 · 「확인」으로 채점

// :root에 주요 색·글자색·배경색·모서리 반경을 변수로 정의하고, h1·button·.card에서 그 변수를 꺼내 쓰세요.

힌트

정의는 --이름: 값; 꺼내 쓰기는 var(--이름) 입니다.

실습 7 모범답안 (1/2)

```
1  :root {
2    --main: #4338ca;
3    --text: #111827;
4    --bg: #ffffff;
5    --radius: 12px;
6  }
7
8  h1 { color: var(--main); }
9
10 button {
11   background: var(--main);
12   color: #fff;
```


실습 7 모범답안 (2/2)

```
19     border: 0;
20     padding: 10px 20px;
21     border-radius: var(--radius);
22 }
23
24 .card {
25     background: var(--bg);
26     color: var(--text);
27     border-radius: var(--radius);
28     padding: 20px;
29 }
```

9주차 마무리

9주차 마무리

CSS3 고급 활용 — 이 시간에 다룬 것을 학습사이트에서 다시 확인합니다

✓ 학습사이트 주차 페이지의 절마다 「이해했어요 · 실습했어요 · 어려워요」 표시

✓ 실습 7개를 실습장에서 「확인」 까지 — 「실습했어요」 체크

✓ 퀴즈 7 「퀴즈 7 — CSS3 고급 활용」 응시 (성적 미반영 · 학습 확인용)

✓ 막히면 「질문있어요」 또는 패들릿에 — 30분 넘게 혼자 막혀 있지 않기

다음 시간

10주차 · JavaScript 기초 문법 이해